

Chapter 15

Dynamic Programming

Masroor Hussain

Professor

Faculty of Computer Science and Engineering

آيَاتُهَا ۶۲ (۵۳) سُورَةُ النَّجْمِ مَكِّيَّةٌ (۲۲) رُكُوعَاتُهَا ۳

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ

شروع اللہ کے نام سے جو بڑا مہربان نہایت رحم کرنے والا ہے

یہ کہ نہیں اٹھاتا کوئی بوجھ اٹھانے والا، بوجھ دوسرے کا ۵۸

اور یہ کہ نہیں مٹا انسان کو مگر وہی کچھ جس کی وہ کوشش کرتا ہے ۵۹

اور یہ کہ اس کی کائنات عظیم سے دکھائی جائے گی ۶۰

اَلَا تَذَرُنْ وَاٰرِثَ رَثِّهٖ وَزَرَ اٰخِرٰی ۶۱

وَ اَنْ لَّيْسَ لِلْاِنْسَانِ اِلَّا مَا سَعٰی ۶۲

وَ اَنْ سَعِيَّهٖ سَوْفَ يُرٰی ۶۳

Dynamic programming

- Like the divide-and-conquer method
 - Divide-and-conquer algorithms partition the problem into disjoint subproblems, solve the subproblems recursively, and then combine their solutions to solve the original problem.
 - Dynamic programming
 - It applies when the subproblems are overlapped, means when subproblems share subsubproblems.
 - Method includes four steps:
 1. Characterize the structure of an optimal solution.
 2. Recursively define the value of an optimal solution.
 3. Compute the value of an optimal solution, typically in a bottom-up fashion.
 4. Construct an optimal solution from computed information.
-

Rod Cutting Example

- The price p_i in dollars that Serling Enterprises charges for a rod of length i inches.
- The ***rod-cutting problem is the following.*** Given a rod of length n inches and a table of prices p_i for $i = 1, 2, \dots, n$, determine the maximum revenue r_n obtainable by cutting up the rod and selling the pieces.
- We can cut up a rod of length n in $2^n - 1$ different ways

length i	1	2	3	4	5	6	7	8	9	10
price p_i	1	5	8	9	10	17	17	20	24	30

Rod cutting example ...

- We see that cutting a 4-inch rod into two 2-inch pieces produces revenue $p_2 + p_2 = 5 + 5 = 10$, which is optimal.

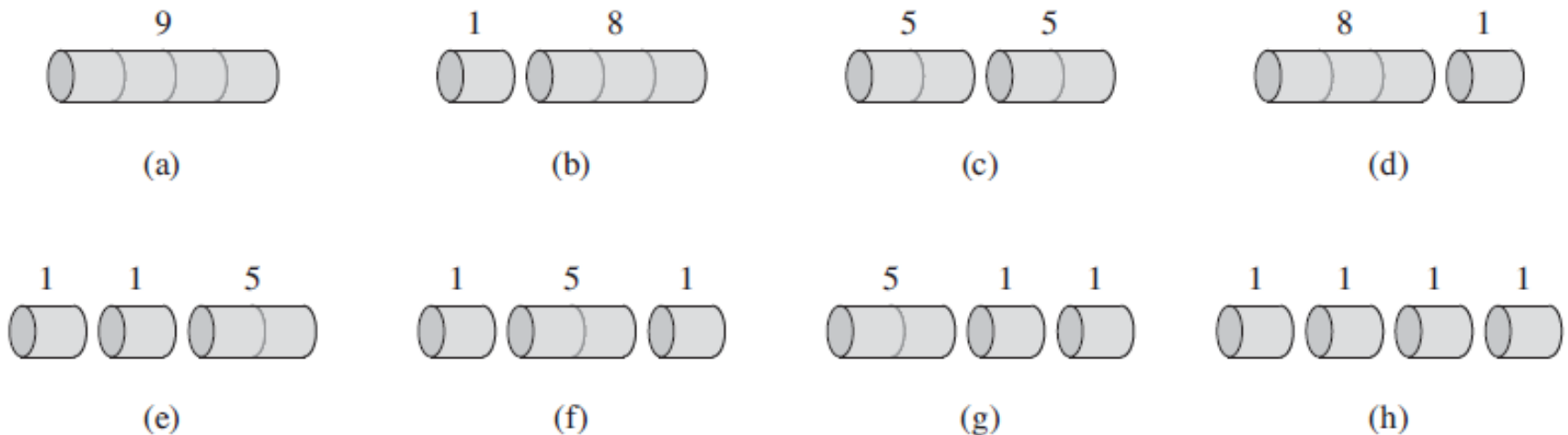


Figure 15.2 The 8 possible ways of cutting up a rod of length 4. Above each piece is the value of that piece, according to the sample price chart of Figure 15.1. The optimal strategy is part (c)—cutting the rod into two pieces of length 2—which has total value 10.

Rod Cutting Example ...

$$n = i_1 + i_2 + \cdots + i_k$$

$$r_n = p_{i_1} + p_{i_2} + \cdots + p_{i_k}$$

$$r_1 = 1 \quad \text{from solution } 1 = 1 \quad (\text{no cuts}),$$

$$r_2 = 5 \quad \text{from solution } 2 = 2 \quad (\text{no cuts}),$$

$$r_3 = 8 \quad \text{from solution } 3 = 3 \quad (\text{no cuts}),$$

$$r_n = \max (p_n, r_1 + r_{n-1}, r_2 + r_{n-2}, \dots, r_{n-1} + r_1)$$

$$r_6 = 17 \quad \text{from solution } 6 = 6 \quad (\text{no cuts}),$$

$$r_7 = 18 \quad \text{from solution } 7 = 1 + 6 \quad \text{or } 7 = 2 + 2 + 3,$$

$$r_8 = 22 \quad \text{from } r_n = \max_{1 \leq i \leq n} (p_i + r_{n-i})$$

$$r_9 = 25 \quad \text{from}$$

$$r_{10} = 30 \quad \text{from solution } 10 = 10 \quad (\text{no cuts}).$$

Recursive Algorithm

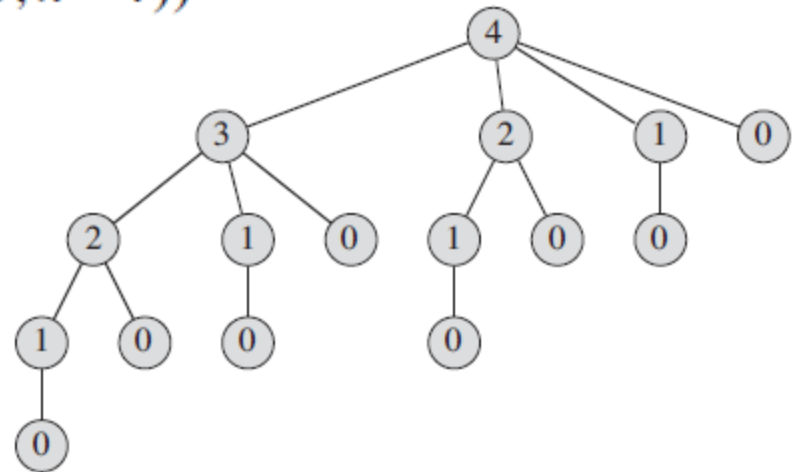
CUT-ROD(p, n)

```
1  if  $n == 0$ 
2      return 0
3   $q = -\infty$ 
4  for  $i = 1$  to  $n$ 
5       $q = \max(q, p[i] + \text{CUT-ROD}(p, n - i))$ 
6  return  $q$ 
```

$$T(n) = 1 + \sum_{j=0}^{n-1} T(j)$$

$$T(n) = 2^n$$

$$r_n = \max_{1 \leq i \leq n} (p_i + r_{n-i})$$



Dynamic Programming

- To-down with memoization
- Bottom up method

Top-down with memoization

MEMOIZED-CUT-ROD(p, n)

```
1  let  $r[0..n]$  be a new array
2  for  $i = 0$  to  $n$ 
3       $r[i] = -\infty$ 
4  return MEMOIZED-CUT-ROD-AUX( $p, n, r$ )
```

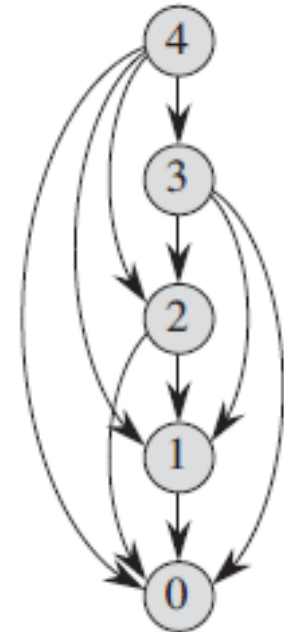
MEMOIZED-CUT-ROD-AUX(p, n, r)

```
1  if  $r[n] \geq 0$ 
2      return  $r[n]$ 
3  if  $n == 0$ 
4       $q = 0$ 
5  else  $q = -\infty$ 
6      for  $i = 1$  to  $n$ 
7           $q = \max(q, p[i] + \text{MEMOIZED-CUT-ROD-AUX}(p, n - i, r))$ 
8   $r[n] = q$ 
9  return  $q$ 
```

Bottom up

BOTTOM-UP-CUT-ROD(p, n)

```
1  let  $r[0..n]$  be a new array
2   $r[0] = 0$ 
3  for  $j = 1$  to  $n$ 
4       $q = -\infty$ 
5      for  $i = 1$  to  $j$ 
6           $q = \max(q, p[i] + r[j - i])$ 
7       $r[j] = q$ 
8  return  $r[n]$ 
```



The running time of procedure **BOTTOM-UP-CUT-ROD** is $\Theta(n^2)$

Reconstruction of the solution

EXTENDED-BOTTOM-UP-CUT-ROD(p, n)

```
1  let  $r[0..n]$  and  $s[0..n]$  be new arrays
2   $r[0] = 0$ 
3  for  $j = 1$  to  $n$ 
4       $q = -\infty$ 
5      for  $i = 1$  to  $j$ 
6          if  $q < p[i] + r[j - i]$ 
7               $q = p[i] + r[j - i]$ 
8               $s[j] = i$ 
9       $r[j] = q$ 
10 return  $r$  and  $s$ 
```

i	0	1	2	3	4	5	6	7	8	9	10
$r[i]$	0	1	5	8	10	13	17	18	22	25	30
$s[i]$	0	1	2	3	2	2	6	1	2	3	10

Print the solution

PRINT-CUT-ROD-SOLUTION(p, n)

```
1  ( $r, s$ ) = EXTENDED-BOTTOM-UP-CUT-ROD( $p, n$ )
2  while  $n > 0$ 
3      print  $s[n]$ 
4       $n = n - s[n]$ 
```

Matrix Multiplication

MATRIX-MULTIPLY (A, B)

```
1  if  $A.columns \neq B.rows$ 
2      error "incompatible dimensions"
3  else let  $C$  be a new  $A.rows \times B.columns$  matrix
4      for  $i = 1$  to  $A.rows$ 
5          for  $j = 1$  to  $B.columns$ 
6               $c_{ij} = 0$ 
7              for  $k = 1$  to  $A.columns$ 
8                   $c_{ij} = c_{ij} + a_{ik} \cdot b_{kj}$ 
9  return  $C$ 
```

Matrix Chain Multiplication

- Given a sequence (chain) $\langle A_1, A_2, \dots, A_n \rangle$ of n matrices to be multiplied
- E.g. Multiplying $A_1 A_2 A_3 A_4$ have five ways:
 - $(A_1(A_2(A_3 A_4)))$
 - $(A_1 ((A_2 A_3) A_4))$
 - $((A_1 A_2)(A_3 A_4))$
 - $((A_1 (A_2 A_3)) A_4)$
 - $((A_1 A_2) A_3) A_4$
- Total: $P(n) = \begin{cases} 1 & \text{if } n = 1, \\ \sum_{k=1}^{n-1} P(k)P(n-k) & \text{if } n \geq 2. \end{cases}$ (Catalan number)

Applying Dynamic Programming

1. Characterize the structure of an optimal solution.
2. Recursively define the value of an optimal solution.
3. Compute the value of an optimal solution.
4. Construct an optimal solution from computed information.

Recursive Solution

$$m[i, j] = \begin{cases} 0 & \text{if } i = j, \\ \min_{i \leq k < j} \{m[i, k] + m[k + 1, j] + p_{i-1}p_kp_j\} & \text{if } i < j. \end{cases}$$

RECURSIVE-MATRIX-CHAIN(p, i, j)

```
1  if  $i == j$ 
2      return 0
3   $m[i, j] = \infty$ 
4  for  $k = i$  to  $j - 1$ 
5       $q = \text{RECURSIVE-MATRIX-CHAIN}(p, i, k)$ 
            $+ \text{RECURSIVE-MATRIX-CHAIN}(p, k + 1, j)$ 
            $+ p_{i-1}p_kp_j$ 
6      if  $q < m[i, j]$ 
7           $m[i, j] = q$ 
8  return  $m[i, j]$ 
```

Computing the optimal costs

MATRIX-CHAIN-ORDER(p)

```
1   $n = p.length - 1$ 
2  let  $m[1..n, 1..n]$  and  $s[1..n - 1, 2..n]$  be new tables
3  for  $i = 1$  to  $n$ 
4       $m[i, i] = 0$ 
5  for  $l = 2$  to  $n$            //  $l$  is the chain length
6      for  $i = 1$  to  $n - l + 1$ 
7           $j = i + l - 1$ 
8           $m[i, j] = \infty$ 
9          for  $k = i$  to  $j - 1$ 
10              $q = m[i, k] + m[k + 1, j] + p_{i-1}p_kp_j$ 
11             if  $q < m[i, j]$ 
12                  $m[i, j] = q$ 
13                  $s[i, j] = k$ 
14  return  $m$  and  $s$ 
```

Computing the optimal costs...

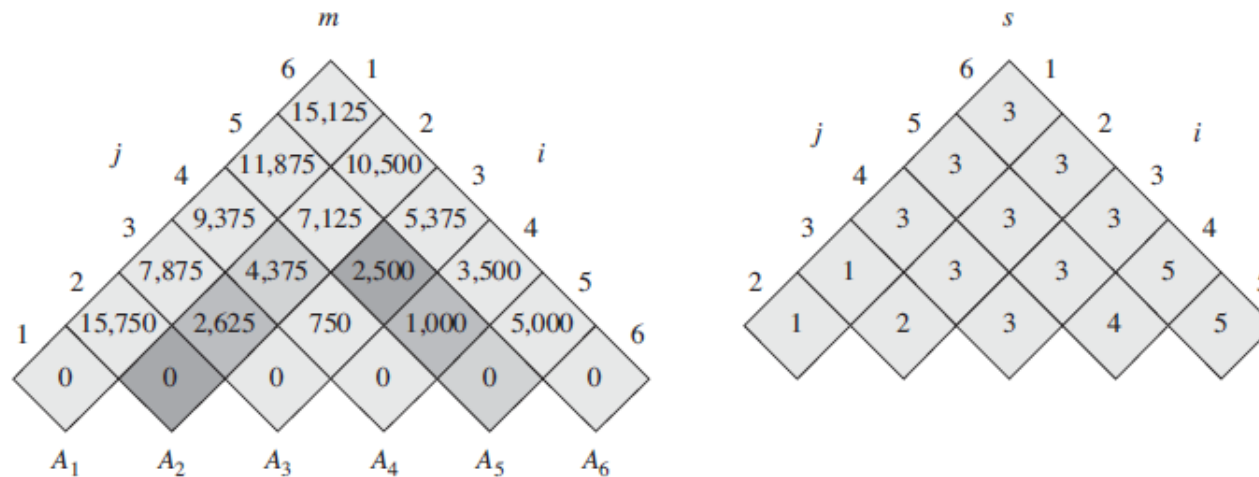


Figure 15.5 The m and s tables computed by MATRIX-CHAIN-ORDER for $n = 6$ and the following matrix dimensions:

matrix	A_1	A_2	A_3	A_4	A_5	A_6
dimension	30×35	35×15	15×5	5×10	10×20	20×25

The tables are rotated so that the main diagonal runs horizontally. The m table uses only the main diagonal and upper triangle, and the s table uses only the upper triangle. The minimum number of scalar multiplications to multiply the 6 matrices is $m[1, 6] = 15,125$. Of the darker entries, the pairs that have the same shading are taken together in line 10 when computing

$$m[2, 5] = \min \begin{cases} m[2, 2] + m[3, 5] + p_1 p_2 p_5 = 0 + 2500 + 35 \cdot 15 \cdot 20 = 13,000, \\ m[2, 3] + m[4, 5] + p_1 p_3 p_5 = 2625 + 1000 + 35 \cdot 5 \cdot 20 = 7125, \\ m[2, 4] + m[5, 5] + p_1 p_4 p_5 = 4375 + 0 + 35 \cdot 10 \cdot 20 = 11,375 \end{cases} \\ = 7125.$$

Constructing an optimal solution

PRINT-OPTIMAL-PARENS (s, i, j)

```
1  if  $i == j$ 
2      print " $A$ " $i$ 
3  else print "("
4      PRINT-OPTIMAL-PARENS ( $s, i, s[i, j]$ )
5      PRINT-OPTIMAL-PARENS ( $s, s[i, j] + 1, j$ )
6      print ")"
```

Subtleties

- **Unweighted shortest path:**
 - Find a path from u to v consisting of the fewest edges. Such a path must be simple, since removing a cycle from a path produces a path with fewer edges.
- **Unweighted longest simple path:**
 - Find a simple path from u to v consisting of the most edges. We need to include the requirement of simplicity because otherwise we can traverse a cycle as many times as we like to create paths with an arbitrarily large number of edges.

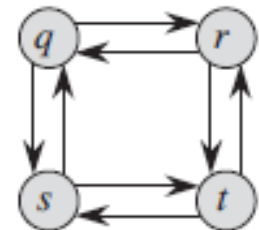
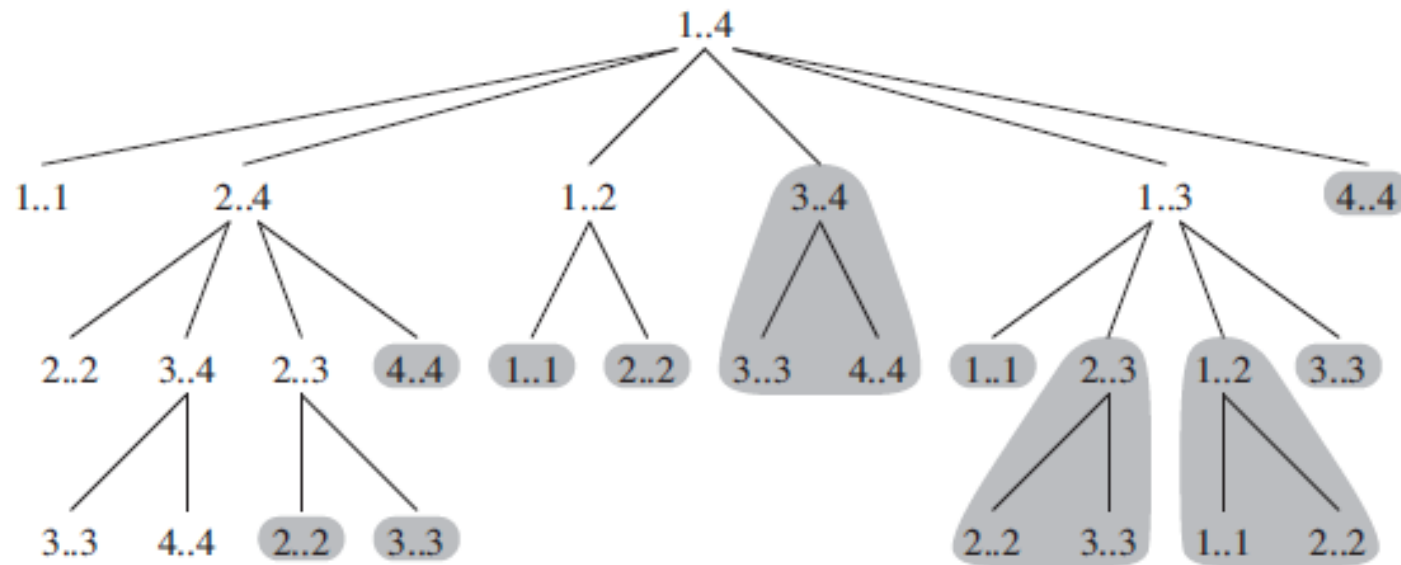


Figure 15.6 A directed graph showing that the problem of finding a longest simple path in an unweighted directed graph does not have optimal substructure. The path $q \rightarrow r \rightarrow t$ is a longest simple path from q to t , but the subpath $q \rightarrow r$ is not a longest simple path from q to r , nor is the subpath $r \rightarrow t$ a longest simple path from r to t .

Overlapping of subproblems



Recursive Solution

RECURSIVE-MATRIX-CHAIN(p, i, j)

```
1  if  $i == j$ 
2      return 0
3   $m[i, j] = \infty$ 
4  for  $k = i$  to  $j - 1$ 
5       $q = \text{RECURSIVE-MATRIX-CHAIN}(p, i, k)$ 
          +  $\text{RECURSIVE-MATRIX-CHAIN}(p, k + 1, j)$ 
          +  $p_{i-1}p_kp_j$ 
6      if  $q < m[i, j]$ 
7           $m[i, j] = q$ 
8  return  $m[i, j]$ 
```

$$T(1) \geq 1,$$

$$T(n) \geq 1 + \sum_{k=1}^{n-1} (T(k) + T(n-k) + 1) \quad \text{for } n > 1.$$

Recursive time complexity

$$T(n) \geq 2 \sum_{i=1}^{n-1} T(i) + n$$

Prove
that

$$T(n) = \Omega(2^n)$$

$$T(1) \geq 1 = 2^0$$

We shall show that $T(n) \geq 2^{n-1}$

$$\begin{aligned} T(n) &\geq 2 \sum_{i=1}^{n-1} 2^{i-1} + n &= 2(2^{n-1} - 1) + n \\ &= 2 \sum_{i=0}^{n-2} 2^i + n &= 2^n - 2 + n \\ &&\geq 2^{n-1}, \end{aligned}$$


Memoization in $O(n^3)$

MEMOIZED-MATRIX-CHAIN(p)

```
1   $n = p.length - 1$ 
2  let  $m[1..n, 1..n]$  be a new table
3  for  $i = 1$  to  $n$ 
4      for  $j = i$  to  $n$ 
5           $m[i, j] = \infty$ 
6  return LOOKUP-CHAIN( $m, p, 1, n$ )
```

LOOKUP-CHAIN(m, p, i, j)

```
1  if  $m[i, j] < \infty$ 
2      return  $m[i, j]$ 
3  if  $i == j$ 
4       $m[i, j] = 0$ 
5  else for  $k = i$  to  $j - 1$ 
6       $q = \text{LOOKUP-CHAIN}(m, p, i, k)$ 
         $+ \text{LOOKUP-CHAIN}(m, p, k + 1, j) + p_{i-1}p_kp_j$ 
7      if  $q < m[i, j]$ 
8           $m[i, j] = q$ 
9  return  $m[i, j]$ 
```

Longest common sub-sequence

- Step 1: Characterizing a longest common subsequence

Theorem 15.1 (Optimal substructure of an LCS)

Let $X = \langle x_1, x_2, \dots, x_m \rangle$ and $Y = \langle y_1, y_2, \dots, y_n \rangle$ be sequences, and let $Z = \langle z_1, z_2, \dots, z_k \rangle$ be any LCS of X and Y .

1. If $x_m = y_n$, then $z_k = x_m = y_n$ and Z_{k-1} is an LCS of X_{m-1} and Y_{n-1} .
2. If $x_m \neq y_n$, then $z_k \neq x_m$ implies that Z is an LCS of X_{m-1} and Y .
3. If $x_m \neq y_n$, then $z_k \neq y_n$ implies that Z is an LCS of X and Y_{n-1} .

Step 2: A recursive solution

$$c[i, j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0, \\ c[i - 1, j - 1] + 1 & \text{if } i, j > 0 \text{ and } x_i = y_j, \\ \max(c[i, j - 1], c[i - 1, j]) & \text{if } i, j > 0 \text{ and } x_i \neq y_j. \end{cases}$$

Step 3: Computing the length of an LCS

LCS-LENGTH(X, Y)

```
1   $m = X.length$ 
2   $n = Y.length$ 
3  let  $b[1..m, 1..n]$  and  $c[0..m, 0..n]$  be new tables
4  for  $i = 1$  to  $m$ 
5       $c[i, 0] = 0$ 
6  for  $j = 0$  to  $n$ 
7       $c[0, j] = 0$ 
8  for  $i = 1$  to  $m$ 
9      for  $j = 1$  to  $n$ 
10         if  $x_i == y_j$ 
11              $c[i, j] = c[i - 1, j - 1] + 1$ 
12              $b[i, j] = \nwarrow$ 
13         elseif  $c[i - 1, j] \geq c[i, j - 1]$ 
14              $c[i, j] = c[i - 1, j]$ 
15              $b[i, j] = \uparrow$ 
16         else  $c[i, j] = c[i, j - 1]$ 
17              $b[i, j] = \leftarrow$ 
18  return  $c$  and  $b$ 
```

Step 4: Constructing an LCS in $O(m + n)$

		j	0	1	2	3	4	5	6
			y_j B D C A B A						
i	x_i								
0			0	0	0	0	0	0	0
1	A		0	↑	↑	↑	↖1	←1	↖1
2	B		0	↖1	←1	←1	↑1	↖2	←2
3	C		0	↑1	↑1	↖2	←2	↑2	↑2
4	B		0	↖1	↑1	↑2	↑2	↖3	←3
5	D		0	↑1	↖2	↑2	↑2	↑3	↑3
6	A		0	↑1	↑2	↑2	↖3	↑3	↖4
7	B		0	↖1	↑2	↑2	↑3	↖4	↑4

Figure 15.8 The c and b tables computed by LCS-LENGTH on the sequences $X = \langle A, B, C, B, D, A, B \rangle$ and $Y = \langle B, D, C, A, B, A \rangle$. The square in row i and column j contains the value of $c[i, j]$ and the appropriate arrow for the value of $b[i, j]$. The entry 4 in $c[7, 6]$ —the lower right-hand corner of the table—is the length of an LCS $\langle B, C, B, A \rangle$ of X and Y . For $i, j > 0$, entry $c[i, j]$ depends only on whether $x_i = y_j$ and the values in entries $c[i - 1, j]$, $c[i, j - 1]$, and $c[i - 1, j - 1]$, which are computed before $c[i, j]$. To reconstruct the elements of an LCS, follow the $b[i, j]$ arrows from the lower right-hand corner; the sequence is shaded. Each “↖” on the shaded sequence corresponds to an entry (highlighted) for which $x_i = y_j$ is a member of an LCS.

Printing the solution

PRINT-LCS(b, X, i, j)

1 if $i == 0$ or $j == 0$

2 return

3 if $b[i, j] == \nwarrow$

4 PRINT-LCS($b, X, i - 1, j - 1$)

5 print x_i

6 elseif $b[i, j] == \uparrow$

7 PRINT-LCS($b, X, i - 1, j$)

8 else PRINT-LCS($b, X, i, j - 1$)